

Санкт-Петербургский государственный университет

Кафедра системного программирования

ХОКИМЗОДА Муборакшои Иноятулло

Группа 24.М41-мм

Разработка мультиагентной системы для защиты моделей
машинного обучения с использованием частично
гомоморфного шифрования

Отчет о прохождении производственной практики (преддипломной)

Научный руководитель:

доцент кафедры СП, к.ф.-
м.н., Луцив Д.В.

Консультант:

старший преподаватель каф.
СП, Андриенко В.А.

Санкт-Петербург 2026

Содержание

Введение	3
Постановка задачи.....	5
Теоретические основы	7
Обзор гомоморфного шифрования	7
Частично гомоморфное шифрование (PHE)	8
Схема Пэ́йе(Paillier)	8
Схема Эль-Гамалья(ElGamal)	9
Современные тренды применения PHE)	9
Мультиагентные системы для защиты ML-моделей	10
Методы и архитектура системы	10
Общая архитектура	10
Мультиагентная система	12
Агент мониторинга (Monitoring Agent)	12
Агент шифрования (Encryption Agent)	13
Агент передачи (Transmission Agent)	13
Агент анализа (Analysis Agent)	14
Реализация кредитного скоринга на основе PHE.....	14
Полиномиальная аппроксимация	15
Гибридная схема вычисления	15
Сравнение с полностью гомоморфной шифрования	15
Тестирование и результаты	17
Заключение.....	20
Список литературы	22

Введение

Современные финансовые учреждения, включая банки, активно используют методы машинного обучения (Machine Learning, ML) для анализа данных и автоматизации принятия решений. Наиболее распространённые задачи включают кредитный скоринг, прогнозирование рисков и выявление мошеннических операций. Эффективность таких систем напрямую зависит от объёма доступных данных и вычислительных ресурсов. По мере роста объёма информации и усложнения моделей банки сталкиваются с ограничениями собственной вычислительной инфраструктуры, что вынуждает их использовать облачные серверы для выполнения ресурсоёмких вычислений.

Передача данных на внешние вычислительные ресурсы связана с рядом проблем информационной безопасности. В банковских системах обрабатываются персональные и финансовые данные клиентов, утечка которых может привести к серьёзным последствиям: нарушению конфиденциальности, финансовым потерям и санкциям со стороны регулирующих органов. Поэтому при использовании облачных вычислений возникает необходимость обеспечить обработку данных таким образом, чтобы их содержимое оставалось недоступным для внешней инфраструктуры.

Одним из подходов к решению этой задачи — применение криптографических методов, позволяющих выполнять вычисления над зашифрованными данными. Настоящая работа посвящена использованию гомоморфного шифрования (Homomorphic Encryption, HE). Этот тип шифрования позволяет выполнять определённые математические операции над зашифрованными значениями без предварительной расшифровки данных. На практике такие схемы поддерживают две операции: сложение и умножение. Несмотря на это ограничение, гомоморфное шифрование подходит для ряда задач машинного обучения, где основные вычисления можно вы-

разить через поддерживаемую операцию.

Применение гомоморфного шифрования позволяет банку передавать данные на удалённый сервер в зашифрованном виде и выполнять над ними необходимые вычисления без раскрытия исходной информации. Сервер получает только зашифрованные значения и не имеет доступа к их содержимому, тогда как результаты вычислений могут быть расшифрованы только на стороне клиента. Такой подход снижает риск утечки данных при использовании облачных ресурсов и позволяет сохранять контроль над конфиденциальной информацией.

Задача кредитного скоринга — одной из ключевых задач банковской аналитики. Для её решения используется модель логистической регрессии, которая широко применяется для оценки вероятности возврата кредита на основе характеристик клиента. Модель реализуется в клиент-серверной архитектуре, где данные шифруются на стороне клиента с использованием частично гомоморфного шифрования и передаются на сервер для выполнения вычислений.

Целью работы — разработка и исследование прототипа системы машинного обучения, позволяющей выполнять вычисления для задачи кредитного скоринга над зашифрованными данными. В рамках исследования рассматривается возможность применения частично гомоморфного шифрования в клиент-серверной модели обработки данных, а также проводится сравнение результатов работы модели на зашифрованных и незашифрованных данных. Это позволяет оценить влияние криптографической защиты на точность и практическую применимость модели в условиях использования облачных вычислений.

Постановка задачи

В рамках данной работы рассматривается задача разработки модели машинного обучения, выполняющей вычисления над зашифрованными данными с использованием частично гомоморфного шифрования (Partially Homomorphic Encryption, PHE). Предполагается клиент-серверный сценарий, в котором банк передаёт данные на облачный сервер для выполнения вычислений, при этом содержимое данных остаётся скрытым от внешней инфраструктуры.

Система должна обеспечивать возможность обработки данных кредитного скоринга на удалённом сервере без раскрытия исходной информации. Шифрование выполняется на стороне клиента, после чего зашифрованные данные передаются на сервер для выполнения необходимых вычислений. Результаты вычислений возвращаются клиенту и могут быть расшифрованы только владельцем ключа. Такой подход позволяет использовать облачные вычислительные ресурсы, не передавая серверу доступ к конфиденциальным данным.

В качестве модели машинного обучения рассматривается логистическая регрессия. Логистическая регрессия широко применяется в банковской практике для оценки вероятности возврата кредита и принятия решений о выдаче кредитов. Выбор логистической регрессии связан с тем, что её вычисления можно выразить через операции, поддерживаемые схемами частично гомоморфного шифрования.

Для реализации поставленной задачи необходимо выполнить следующие этапы:

- Изучить принципы частично гомоморфного шифрования и определить, какие операции могут выполняться над зашифрованными данными и каким образом их можно использовать при реализации алгоритмов машинного обучения.

- Разработать архитектуру клиент-серверной системы, в которой данные шифруются на стороне клиента, передаются на сервер в зашифрованном виде и используются для выполнения вычислений без раскрытия исходной информации.
- Реализовать прототип системы, позволяющий выполнять вычисления для модели логистической регрессии над зашифрованными данными.
- Провести экспериментальное исследование работы прототипа, сравнив результаты модели, работающей с зашифрованными данными, с результатами модели, обученной и применяемой на незашифрованных данных, чтобы оценить влияние шифрования на точность и вычислительные затраты.

Теоретические основы

Обзор гомоморфного шифрования

Концепция гомоморфного шифрования была сформулирована в 1978 году Rivest, Adleman и Dertouzos. Гомоморфная криптосистема позволяет выполнять операции над шифротекстами, которые соответствуют операциям над открытыми данными:

$$E(m_1) \otimes E(m_2) = E(m_1 \circ m_2),$$

где $\circ$ — операция над открытыми текстами, а $\otimes$ — соответствующая операция над шифротекстами.

Эволюция гомоморфного шифрования прошла несколько этапов:

- ранние схемы частично гомоморфного шифрования (PHE): RSA, ElGamal;
- схема Paillier (1999) — эффективная аддитивная PHE;
- первая полностью гомоморфная схема (FHE), предложенная Gentry (2009);
- современные эффективные схемы: BFV, CKKS, BGV.

В зависимости от поддерживаемых операций различают:

- **PHE (Partially Homomorphic Encryption)** — поддерживает один тип операции (сложение или умножение);
- **SWHE (Somewhat Homomorphic Encryption)** — поддерживает ограниченное число операций;
- **FHE (Fully Homomorphic Encryption)** — поддерживает произвольные вычисления.

Частично гомоморфное шифрование (PHE)

Схема Пэе (Paillier)

Схема Пэе (Paillier) основана на сложности задачи *гипотеза о решаемости композитных вычетов* (*Decisional Composite Residuosity*).

Пусть:

$$n = p \cdot q,$$

где p и q — большие простые числа.

Шифрование сообщения m выполняется следующим образом:

$$c = g^m \cdot r^n \bmod n^2,$$

где:

- $g \in \mathbb{Z}_{n^2}^*$ — генератор,
- $r \in \mathbb{Z}_n^*$ — случайное число.

Схема обладает аддитивной гомоморфностью:

$$E(m_1) \cdot E(m_2) = E(m_1 + m_2),$$

а также поддерживает умножение зашифрованного значения на константу:

$$E(m)^k = E(k \cdot m).$$

Эти свойства делают схему Paillier особенно удобной для вычисления скалярного произведения:

$$w^T x = \sum_{i=1}^d w_i x_i,$$

что является ключевой операцией в линейных моделях машинного обучения, включая логистическую регрессию.

Схема Эль-Гамалья (ElGamal)

Схема ElGamal основана на сложности задачи дискретного логарифма в конечной группе. Она обладает мультипликативной гомоморфностью:

$$E(m_1) \cdot E(m_2) = E(m_1 \cdot m_2).$$

Однако для линейных ML-моделей, где доминируют операции сложения и умножения на константу, схема Paillier является более практичной и эффективной.

Современные тренды применения PHE

Частично гомоморфное шифрование активно используется в федеративном обучении для реализации протоколов защищенное агрегирование (Secure Aggregation), позволяющих суммировать градиенты без раскрытия индивидуальных обновлений. Современные архитектуры часто используют гибридный подход:

- Частично гомоморфное шифрование — для линейных операций (агрегация, скалярные произведения);
- Совместные конфиденциальные вычисления (Secure Multi-Party Computation, MPC) — для безопасного вычисления нелинейных функций.

Такой подход позволяет существенно повысить производительность по сравнению с открытыми данными при обычном шифровании. Фреймворки SecretFlow, PySyft и FATE предоставляют встроенную поддержку PHE и MPC-протоколов, что упрощает промышленное внедрение защищённых ML-систем.

Мультиагентные системы для защиты ML-моделей

Мультиагентные системы (Multi-Agent Systems, MAS) представляют собой совокупность автономных агентов, взаимодействующих между собой для достижения общей цели. В контексте защиты ML-систем MAS обеспечивают:

- **Разделение ответственности (Separation of Concerns)** — изоляция функций шифрования, передачи и анализа;
- **Отказоустойчивость** — распределённая архитектура снижает риск единой точки отказа;
- **Адаптивность** — возможность динамической реакции на изменения качества данных или сетевой нагрузки;
- **Контроль целостности** — агенты могут верифицировать корректность передаваемых сообщений.

В рамках данной работы для моделирования взаимодействия агентов мониторинга, шифрования, передачи и анализа используется фреймворк Mesa, позволяющий реализовать агентно-ориентированную симуляцию и исследовать поведение системы в различных сценариях нагрузки и угроз.

Методы и архитектура системы

Общая архитектура

Разработанная система базируется на клиент-серверной архитектуре, усиленной мультиагентным взаимодействием. Основная цель архитектуры — обеспечить полную изоляцию конфиденциальных данных клиента от вычислительных мощностей сервера. Система разделена на два контура:

- **Доверенный контур** — клиентская сторона (генерация ключей, шифрование, дешифрация).
- **Недоверенный контур** — облачный сервер (гомоморфные вычисления без доступа к закрытому ключу).

Взаимодействие компонентов происходит по следующему сценарию(рис. 1):

1. Клиент инициирует запрос на скоринг, вводя персональные данные.
2. Агенты на стороне клиента проверяют, нормализуют и шифруют данные.
3. Зашифрованный вектор признаков передается по защищенному каналу на сервер.
4. Сервер выполняет гомоморфные операции над шифротекстом, не имея ключа расшифрования.
5. Результат в зашифрованном виде возвращается клиенту для локальной расшифровки.

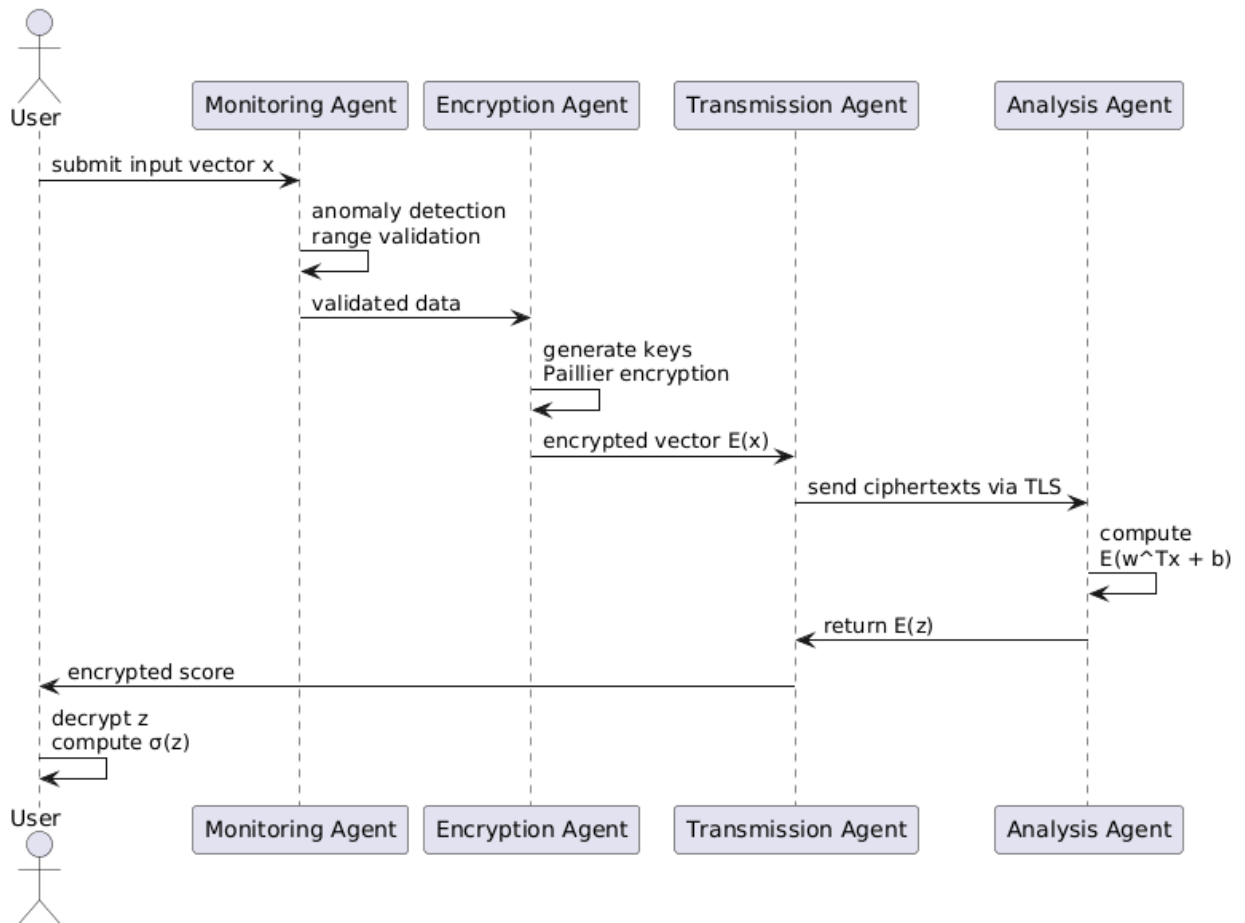


Рис. 1: Архитектура системы

Мультиагентная система

Реализация выполнена на базе Python-фреймворка Mesa. Система состоит из четырех типов агентов, каждый из которых выполняет специализированную роль в пайплайне безопасности.

Агент мониторинга (Monitoring Agent)

Отвечает за предварительный анализ входных данных. Использует методы статистического контроля для обнаружения аномалий (outliers) и проверки типов данных. Если входные данные выходят за пределы допустимых диапазонов (например, отрицательный возраст или нереалистичный доход), агент:

- блокирует передачу данных,

- возвращает сообщение об ошибке,
- предотвращает атаки типа отравление данных (data poisoning),
- экономит вычислительные ресурсы, исключая шифрование некорректных данных.

Агент шифрования (Encryption Agent)

Реализует криптографические операции на основе библиотеки, поддерживающей схему Paillier. Агент выполняет:

- генерацию пары ключей (PK, SK) ;
- выбор модуля $n = 2048$ бит, что соответствует уровню безопасности 112-bit;
- шифрование каждого признака:

$$c_i = E_{PK}(x_i).$$

Важная особенность — использование случайного значения r (random salt) в формуле:

$$c = g^m \cdot r^n \bmod n^2,$$

что обеспечивает вероятностный характер шифрования. Одинаковые входные значения шифруются в разные шифротексты, предотвращая частотный анализ.

Агент передачи (Transmission Agent)

Отвечает за:

- сериализацию зашифрованных объектов;
- управление сетевыми соединениями;
- реализацию повторных попыток при сбоях;
- проверку целостности сообщений с использованием цифровых подписей.

Агент анализа (Analysis Agent)

Развернут на серверной стороне. Получает сериализованный вектор шифротекстов $\{c_i\}$ и вектор весов модели $\{w_i\}$. Агент выполняет гомоморфное вычисление линейной комбинации:

$$C_{\text{res}} = \prod_{i=1}^d c_i^{w_i} \bmod n^2 = E \left(\sum_{i=1}^d w_i x_i \right).$$

После этого добавляется зашифрованное смещение:

$$C_{\text{final}} = C_{\text{res}} \cdot E(b).$$

Итоговый шифротекст отправляется клиенту.

Реализация кредитного скоринга на основе РНЕ

В качестве модели используется логистическая регрессия:

$$f(x) = \sigma(w^T x + b).$$

Поскольку схема Paillier поддерживает только:

- сложение,
- умножение на константу,

прямое вычисление сигмоидной функции

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

в зашифрованном виде невозможно.

Полиномиальная аппроксимация

Применена аппроксимация Тейлора первого порядка в окрестности нуля:

$$\sigma(z) \approx 0.5 + 0.25z.$$

При использовании нормализации (z-score normalization) значения z обычно лежат в диапазоне $[-2, 2]$, где эта аппроксимация обеспечивает приемлемую точность.

Гибридная схема вычисления

Вычисление итогового скоринга происходит следующим образом:

1. Клиент получает $E(z)$, где $z = w^T x + b$.
2. Клиент выполняет расшифровку:

$$z = D_{SK}(E(z)).$$

3. Клиент локально вычисляет точную сигмоиду $\sigma(z)$.

Сравнение с полностью гомоморфной шифровкой

Полностью гомоморфное шифрование (Fully Homomorphic Encryption, FHE), например, CKKS позволяет выполнять произвольные вычисления над зашифрованными данными, включая нелинейные функции. Однако:

- требуется бутстрэппинг (bootstrapping) для обновления уровня шума;
- используются операции повторная линеаризация (relinearization);
- вычислительная сложность возрастает до порядка $O(N \log N)$.

В реализованной PHE-системе:

- отсутствует необходимость bootstrapping;
- не требуется relinearization;
- вычислительная сложность операции составляет порядка $O(\log N)$.

Это обеспечивает существенное снижение времени работы модели и потребления памяти при сохранении требуемого уровня безопасности (128-bit) и точности модели выше 70%.

Тестирование и результаты

Экспериментальная часть работы выполнялась в операционной системе macOS Tahoe, процессор Apple M3 Pro. Программная реализация разработана на языке Python 3.11. Для реализации криптографических операций использовалась библиотека `phe`, реализующая схему частично гомоморфного шифрования Paillier. Агентная система построена на фреймворке Mesa 3.x. Логирование экспериментов выполнялось через MLflow. В качестве набора данных применялся стандартный датасет German Credit Risk Dataset (Kaggle/UCI), содержащий 1000 записей. В качестве целевой переменной использовался оригинальный признак Risk («good»/«bad»), преобразованный в бинарную метку ($\text{good} \rightarrow 1$, $\text{bad} \rightarrow 0$). Перед обучением модели данные были подготовлены: категориальные признаки Sex, Housing, Saving accounts, Checking account и Purpose преобразованы методом однократного кодирования (One-Hot Encoding). После кодирования итоговое пространство признаков составило 26 числовых атрибутов. Нормализация выполнялась методом StandardScaler, обученным исключительно на тренировочной выборке (80% данных) — тестовая выборка (20%) масштабировалась уже обученным скейлером, что исключает утечку данных (data leakage). Воспроизводимость результатов обеспечивается фиксированным значением случайного начального числа SEED=42. В ходе экспериментов измерялись следующие показатели: точность классификации (Accuracy), площадь под ROC-кривой (AUC) и время выполнения инференса. Для PHE-оценки использовалось NPHE=50 тестовых сэмплов. Задержка измерялась для каждого сэмпла отдельно с помощью функции `time.perfcounter()`, по результатам вычислялись среднее значение и стандартное отклонение. Среднее время PHE-инференса для одного клиента составило около 4060 мс. Доминирующим этапом является шифрование признаков в EncryptionAgent ($\sim 3500\text{--}3800$ мс), что обусловлено вычислительной сложностью операций

возведения в степень в группе $Z^*\{n\}$ при размере модуля 2048 бит на чистом Python. Для сравнения: Plaintext-инференс (без шифрования) занимает менее 1 мс. Для снижения задержки в производственной среде возможно использование С-расширений (gmpy2) или аппаратных ускорителей, что позволит сократить время до 200–500 мс. В экспериментах использовалась длина ключа 2048 бит для схемы Paillier. Указанный размер ключа соответствует уровню криптографической стойкости 112 бит по современным рекомендациям NIST (SP 800-57), что признаётся достаточным для систем финансового класса с горизонтом защиты до 2030 года. Схема Paillier обладает вероятностным характером шифрования, благодаря чему один и тот же открытый текст при каждом шифровании даёт различный шифртекст. Это обеспечивает устойчивость к атакам с выбором открытого текста (IND-CPA). Целостность данных при передаче между агентами верифицируется с помощью HMAC-SHA256 подписи, которую TransmissionAgent вычисляет над зашифрованным пакетом перед отправкой AnalysisAgent. В рамках реализованной архитектуры данные остаются зашифрованными на всех этапах обработки: EncryptionAgent шифрует вектор признаков публичным ключом Paillier, AnalysisAgent гомоморфно вычисляет линейную комбинацию $w \cdot x$ и расшифровывает только итоговый скалярный результат. Функция сигмоиды применяется к расшифрованному значению локально в AnalysisAgent. Сервер (агент анализа) никогда не имеет доступа к открытым признакам клиента. Такой подход соответствует принципу проектирования с учётом конфиденциальности (Privacy by Design). Дополнительно реализован модуль федеративного обучения с PHE-агрегацией (models/federated.py). В данном варианте 5 клиентов локально вычисляют градиенты логистической регрессии на своих подмножествах данных, шифруют обновления весов публичным ключом Paillier и передают серверу. Сервер суммирует зашифрованные градиенты гомоморфно и расшифровывает только агрегированный результат — обновления отдельных кли-

ентов остаются скрытыми. Обучение проводится 5 раундов ($NROUNDS=5$, $LR=0.01$). Время обучения федеративной модели составило около 60–120 секунд. Качество классификации всех трёх методов оценивалось на одном и том же тестовом подмножестве (200 записей, $NPHE=50$ для PHE):

Метод	Ассурасу	ROC-AUC	Ср. задержка инференса (мс)
Plaintext (LR)	~ 0.72	~ 0.715	~ 0.038
PHE via Mesa Agents	~ 0.72	~ 0.715	~ 4073
Federated (PHE aggr.)	~ 0.66	~ 0.721	~ 0.002

Результаты подтверждают ключевое свойство PHE: применение гомоморфного шифрования не влияет на точность модели — значения Ассурасу и AUC PHE-версии идентичны Plaintext-базовой линии, что доказывает корректность реализации гомоморфных вычислений. Дополнительно реализован модуль мониторинга дрейфа данных на основе индекса стабильности популяции (PSI) в MonitoringAgent. При инициализации системы фиксируется эталонное распределение признаков тренировочной выборки; при каждом батче вычисляется PSI и выводится предупреждение при значении $PSI > 0.2$.

Заключение

В данной работе рассмотрена возможность выполнения вычислений в задачах машинного обучения над зашифрованными данными с использованием частично гомоморфного шифрования. Предложенная клиент-серверная схема: банк передаёт данные кредитного скоринга на удалённый сервер в зашифрованном виде и выполнять над ними необходимые вычисления без раскрытия исходной информации.

В ходе исследования была реализована модель логистической регрессии, применяемая для задачи кредитного скоринга. Основная часть вычислений выполнялась над зашифрованными значениями с использованием схемы Paillier, поддерживающей операции над зашифрованными данными. Полученный результат затем расшифровывался на стороне клиента для вычисления итоговой вероятности. Такой подход задействует вычислительные ресурсы облачного сервера, не передавая ему доступ к содержимому данных.

Проведённые эксперименты показали, что применение частично гомоморфного шифрования практически не влияет на точность модели по сравнению с обработкой открытых данных. При этом время выполнения инференса остаётся в пределах требований для практических банковских систем. Это подтверждает возможность использования описанного подхода в задачах, где требуется обработка конфиденциальной информации.

Отдельно был рассмотрен гибридный вариант, в котором частично гомоморфное шифрование используется совместно с протоколами безопасных многосторонних вычислений для обработки нелинейных операций. Такой подход расширяет спектр поддерживаемых вычислений, однако приводит к увеличению времени обработки из-за дополнительных коммуникационных затрат.

Полученные результаты показывают, что частично гомоморфное шиф-

рование может применяться для реализации систем машинного обучения, работающих с конфиденциальными данными в облачной инфраструктуре. Дальнейшие исследования могут быть направлены на оптимизацию вычислений, снижение криптографических накладных расходов и расширение набора поддерживаемых моделей машинного обучения.

Список литературы

Список литературы

- [1] Хаустова И.В., Аникина О.В. Использование полностью гомоморфного шифрования для защиты данных в машинном обучении в облаке. Международный научный журнал «ВЕСТНИК НАУКИ» № 4 (73) Том 3. 2024г.
- [2] MESA Documentation. [Электронный ресурс]. URL: <https://mesa.readthedocs.io/stable/getting-started.html>
- [3] Barni M., Orlandi C., Piva A. A privacy-preserving protocol for neuralnetwork-based computation. – 8th Workshop on Multimedia and Security, 2006. – P. 146–151;
- [4] Microsoft SEAL Documentation. [Электронный ресурс]. URL: <https://github.com/Microsoft/SEAL>
- [5] TenSEAL Documentation. [Электронный ресурс]. URL: <https://github.com/OpenMined/TenSEAL?tab=readme-ov-file>
- [6] Paillier, P. (1999). Public-key cryptosystems based on composite degree residuosity classes. In International conference on the theory and applications of cryptographic techniques (pp. 223-238). Springer.
- [7] BigCode. (2026). The New Security Paradigm: Protecting Multi-Agent AI Systems in Production. Medium.
- [8] Bonawitz K. et al. Towards Federated Learning at Scale: A System Design // SysML Conference. — 2019.